

技术札记 · NETWORKING

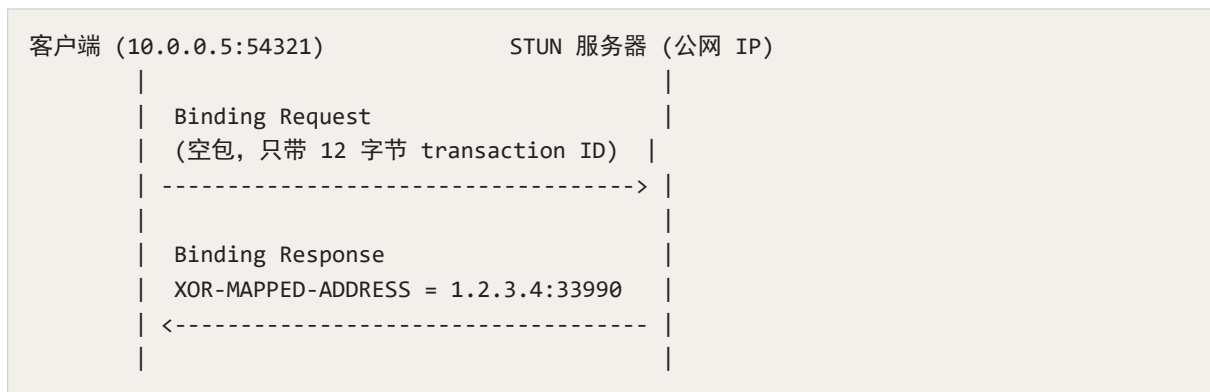
STUN 服务器介绍

从一次最简单的 Binding 交互，到打洞、TURN 中继与 ICE 协作；为什么浏览器只能看到一部分 NAT 行为。

STUN 服务器是什么

STUN (Session Traversal Utilities for NAT, RFC 5389 / 8489) 本质上是一个「告诉我，从你那看我是谁」的工具。它解决一个具体问题：NAT 后面的客户端不知道自己的公网 IP 和端口，因为 NAT 在出包时会把源地址改成自己的公网 IP，把源端口改成一个新分配的端口，客户端自己看不到这个改写结果。

一次最简单的 STUN 交互



- 客户端用一个 真实业务要用的 UDP socket 给 STUN 服务器的 3478 端口发一个 STUN Binding Request。
- 服务端从收到这个包的 IP 数据报上 直接读源 IP / 端口（也就是 NAT 改写后的公网端点），把它塞进响应的 XOR-MAPPED-ADDRESS 属性回传。
- 客户端拿到响应后就知道：「噢，外面看到我是 1.2.3.4:33990」。

关键是这个公网端点对应的是 客户端那个 socket，所以告知对端这个端点后，对端往这里发 UDP 包就能穿透 NAT 到客户端。

报文长什么样

STUN 是非常薄的二进制协议：

20 字节头：
Type(2) Length(2) Magic(4=0x2112A442) Transaction ID(12)
然后是若干 TLV 属性

Type 的高 2 位区分是 request / indication / success response / error response，低 12 位是方法。Magic Cookie 0x2112A442 用来跟早期 STUN 区分版本。Transaction ID 12 字节随机，用来匹配请求和响应。

Binding Response 里你最常看到的属性：

属性名	用途
XOR-MAPPED-ADDRESS	公网端点（XOR 是为了避免某些「智能」NAT 看到自己 IP 后改写返回内容）
MAPPED-ADDRESS	兼容老 RFC 3489 客户端的明文版
SOFTWARE	服务端实现名（可选）
FINGERPRINT	整包 CRC32（可选，防其它协议误判）

整个流程是无连接的，一来一回两个 UDP 包，几十字节。

STUN 只能「告诉」，不能「打洞」

很多人误以为 STUN 服务器会帮我穿透 NAT，不是的。STUN 只回答「你在我这看到的公网端点是什么」。真正打洞的是 ICE 框架——它把 STUN 探测到的候选端点（host / srflx / relay）发给对端，两边同时朝彼此的端点发 UDP 包，靠这种「同时双向打」让两侧的 NAT 都为对方建立映射条目，之后数据流就能直接走 P2P。

如果 NAT 不配合（典型是 Symmetric NAT），打洞失败，ICE 就会回退到 TURN——一个把流量代理转发的中继服务（RFC 8656）。TURN 用同样的端口 3478，协议是 STUN 的超集。

STUN 服务器在判断 NAT 类型时还有这些用途

经典 RFC 3489 的 NAT 类型分类需要 STUN 服务器有两个公网 IP + 两个端口（共 4 个端点）。客户端通过让 STUN 从不同 IP / 端口主动给自己发包，看哪些能收到，区分 Full Cone / Restricted / Port Restricted / Symmetric。RFC 5780 标准化了这套测试。

实务里现在更看重的是 Mapping 行为（同一 socket 给不同目标，公网端点是否变）和 Filtering 行为（不同来源能否回包），不再死守 4 类分法。浏览器（WebRTC）只能做前者。

流量大概多大

一次 binding 总共两个包，100 ~ 200 字节。即便保活心跳 30 秒一次，一个客户端一天也就几 KB。所以公开 STUN 服务能由各家免费托管几乎不影响成本，你看到的小米、B 站、Google、Cloudflare 都开着公开 STUN。

自托管 vs 公开 STUN

维度	公开 STUN（如 stun.miwifi.com）	自托管 STUN
部署成本	0	一台公网 VPS + 3478/udp
国内可达性	看运营商，腾讯 / 小米 / B 站较稳，Google 基本不通	你自己开的，可控
多 IP 支持	大多只有 1 个公网 IP，不能严格判 NAT 类型	要严格判的话需要两网卡或一台机绑两个 IP
隐私	STUN 服务器能记到你的公网端点	自己机器，自己看
速度	通常 < 50ms	看部署位置

shuai-tunnel 服务端自带一个内置 STUN / TURN-lite（监听 3478 / 3479，由 TUNNEL_PEER_MESH_STUN_TURN_PORT 控制），这样客户端做 peer mesh 时可以不依赖任何公开服务，自家全链路闭环。它同时也是 TURN 兜底（直连失败时中继转发加密 UDP）。

常见公开 STUN 服务器

国内可达较稳定

服务地址	说明
stun.miwifi.com:3478	小米路由器走的，国内连通性最好之一
stun.chat.bilibili.com:3478	B 站直播 WebRTC 用的
stun.cloudflare.com:3478	Cloudflare 提供，国内连通性看运营商
stun.nextcloud.com:3478	Nextcloud Talk 用的

国外（国内一般要绕路或不通）

服务地址	说明
stun.l.google.com:19302	注意 Google 用的非标准端口 19302
stun.cloudflare.com:3478	双栈 IPv4 / IPv6
stun.stunprotocol.org:3478	—
stun.voipgate.com:3478	—

不可达（实测）

服务地址	说明
stun.qq.com:3478	国内大量出口超时，已停止服务或被封
stun.douyu.com / douyucdn.cn	同上
stun.syncthing.net	国内出口超时

STUN URI 语法

WebRTC 配置里的 iceServers 字符串严格遵循 RFC 7064:

stun:host[:port]	← 无 TLS
stuns:host[:port]	← TLS over TCP, 少见
turn:host[:port]?transport=udp	← TURN 中继
turns:host[:port]?transport=tcp	← TURN over TLS

默认端口都是 3478（TLS 是 5349）。Google 那个 19302 是个例外。

安全方面

- STUN binding 本身没鉴权，任何人都能查公网端点 —— 这是设计如此，否则没法 P2P。
- TURN allocation 必须用 long-term credential（用户名 / 密码 → HMAC-SHA1 message integrity），不然就是免费代理给全世界用，会被拿去做反射放大攻击或薅流量。
- 公开 STUN 上有放大攻击的历史：源 IP 伪造 + STUN 响应比请求大（带 XOR-MAPPED-ADDRESS 等属性），可以做几倍放大。现在主流实现都对响应大小做了限制，自己开 STUN 时要注意。

跟我们这次 NAT 检测的关系

我们 NAT 检测页面就是用浏览器 WebRTC 创建一个 `PeerConnection`，挂上几个 STUN URL，让它各发一次 `Binding`，从返回的 `srflx candidate` 里读出公网端点。两个 STUN 看到的端点一致 → `Cone-like NAT`；不一致 → `Symmetric NAT`。这就是 STUN 在真实应用里的典型用法。

如果想深入，推荐三份材料

- RFC 8489 —— STUN 协议主文档（更新了 RFC 5389）
- RFC 8445 —— ICE 框架（怎么用 STUN 候选打洞）
- Tailscale 的 `How NAT traversal works` —— 我们帮助文档也是基于这篇做的改写

本文档由 shuai-tunnel 项目整理生成 · 仅作参考